

SSAFY

Vue

Basic Syntax 02

Python



목 차

Computed Property

- Computed
- Computed vs. Methods

Conditional Rendering

- v-if
- v-if vs. v-show

List Rendering

- v-for
- v-for with key
- v-for with v-if

목 차

Watchers

- watch
- computed vs. watch

Lifecycle Hooks

- Lifecycle Hooks 활용

Vue Style Guide

목 차

참고

- computed 주의사항
- Lifecycle Hooks 주의사항
- v-for와 배열을 활용한 “필터링/정렬”
- 배열 변경 관련 메서드
- Todo 애플리케이션 구현



Vue의 다양한 기능들은 어떤 역할을 할까요?

- 아래 퀴즈를 풀어봅시다.

1. 조건에 따라 요소를 DOM에서 완전히 제거 →
2. 조건에 따라 요소를 보이거나 숨기기만 함 →
3. 배열 데이터를 기반으로 목록을 반복하여 렌더링 →
4. 데이터가 바뀔 때마다 새로운 값을 계산하고 결과를 기억(캐싱) →
5. 데이터가 바뀔 때마다 특정 행동(Side Effect)을 수행 →

- 이번 시간을 통해 Vue의 다양한 기능들을 학습하고, 위 퀴즈를 풀어봅시다.

학습 목표

- ✓ computed를 사용해 캐싱되는 계산된 속성을 만들 수 있다.
- ✓ v-if와 v-show의 차이를 알고 조건부 렌더링을 구현한다.
- ✓ v-for 디렉티브를 key와 함께 사용해 목록을 렌더링한다.
- ✓ computed 속성을 활용하여 v-for 렌더링 목록을 필터링한다.
- ✓ watch를 사용해 데이터 변경을 감지하고 특정 작업을 수행한다.
- ✓ computed와 watch의 차이점을 알고 상황에 맞게 사용한다.
- ✓ onMounted 같은 라이프사이클 훅을 사용해 코드를 실행한다.

Computed Properties

Computed

 computed()

computed()

“계산된 속성”을 정의하는 함수

미리 계산된 속성을 만들어 템플릿의 표현식을 단순하게 하고, 불필요한 반복 연산을 줄여줍니다.
한 번 계산된 값은 캐싱(임시 저장)되어, 의존하는 데이터가 바뀌기 전까지는 다시 계산하지 않으므로
성능에 매우 유리합니다.

Computed 가 없는 경우

- 할 일이 남았는지 여부에 따라 다른 메시지를 출력하기

```
<!-- computed.html -->
```

```
const todos = ref([
  { text: 'Vue 실습' },
  { text: '자격증 공부' },
  { text: 'TIL 작성' }
])
```

```
<h2>남은 할 일</h2>
```

```
<p>{{ todos.length > 0 ? '아직 남았다' : '퇴근!' }}</p>
```

- 템플릿이 복잡해지며 todos에 따라 계산을 수행하게 됨
- 만약 이 계산을 템플릿에 여러 번 사용하는 경우에는 매번 계산이 발생

Computed를 사용하는 경우

- 반응형 데이터를 포함하는 복잡한 로직의 경우 computed를 활용하여 미리 값을 계산하여 계산된 값을 사용
- 여러 곳에서 사용해야 한다면, computed로 정의된 restOfTodos를 필요한 곳마다 재사용하면 됨

```
const { createApp, ref, computed } = Vue
```

```
const restOfTodos = computed(() => {  
  return todos.value.length > 0 ? '아직 남았다' : '퇴근!'  
})
```

```
<h2>남은 할 일</h2>  
<p>{{ restOfTodos }}</p>
```


computed 특징

- 반환되는 값은 계산된 ref(computed ref)이며, 일반 ref와 유사하게 계산된 결과를 .value로 참조 가능(템플릿에서는 .value 생략 가능)
- computed 속성은 의존된 반응형 데이터를 **자동으로 추적**
- 의존하는 반응형 데이터가 **변경될 때만 재평가**
 - restOfTodos의 계산은 todos에 의존하고 있음
 - 따라서 todos가 변경될 때만 restOfTodos가 업데이트 됨

```
const restOfTodos = computed(() => {  
  return todos.value.length > 0 ? '아직 남았다' : '퇴근!'  
})
```



ref

기본형 데이터를 반응형으로 만드는 Vue 함수

<개념1_Vue_Basic Syntax 02_ref>



computed ref

원본 데이터가 바뀔 때만, 값을 알아서 다시 계산하는 ref

<개념2_Vue_Basic Syntax 02_computed ref>

Computed vs. Methods

computed와 동일한 로직을 처리할 수 있는 method

- computed 속성 대신 method로도 동일한 기능을 정의할 수 있음

<!-- computed 예시-->

```
const { createApp, ref, computed } = Vue
```

```
const restOfTodos = computed(() => {  
  return todos.value.length > 0 ? '아직 남았다' : '퇴근!'  
})
```

<!-- method 예시-->

```
const getRestOfTodos = function () {  
  return todos.value.length > 0 ? '아직 남았다' : '퇴근!'  
}
```

<h2>남은 할 일</h2>

<p>{{ restOfTodos }}</p>

<p>{{ getRestOfTodos() }}</p>

computed와 method 차이

- computed 속성은 **의존하는 반응형 데이터를 기반으로 그 결과를 캐시(cached)**
 - 의존하는 데이터가 변경된 경우에만 재평가됨
 - 의존하는 데이터가 변경되지 않는 한,
해당 computed 속성에 여러 번 접근해도 함수를 다시 실행하지 않고 캐시된 결과를 즉시 반환
- ※ 반면, method 호출은 다시 렌더링이 발생할 때마다 항상 함수를 실행



캐시(cached)

데이터를 임시로 저장하여
다음에 같은 데이터를 요청할 때
더 빠르게 접근할 수 있도록 하는
기술이나 저장 공간

<개념3_Vue_Basic Syntax 02_캐시(cached)>

TIP

- 템플릿에서 computed는 괄호 없이, method는 괄호를 붙여 호출합니다.
- 계산에 인자가 필요하다면, computed가 아닌 method를 사용해야 합니다.
(계산에 외부의 값이 필요한지 여부를 판단)

캐시

캐시

Cache

데이터나 결과를 일시적으로 저장
해두는 임시 저장소

캐시는 자주 꺼내 먹는 식재료를 넣어두는 '냉장고'와 같은 임시 저장 공간입니다.

요리할 때마다 매번 마트(원본 데이터)에 갈 필요 없이, 냉장에서 바로 재료를 꺼내 쓰니 시간을 크게 절약할 수 있습니다.

마찬가지로 데이터를 요청할 때 먼저 캐시를 확인하고, 없을 경우에만 원본 데이터에 접근하여 가져온 뒤 캐시에 저장합니다.

Cache 예시: 웹 페이지의 캐시 데이터

- 과거 방문한 적이 있는 페이지에 다시 접속할 경우
- 페이지 일부 데이터를 브라우저 **캐시에 저장** 후 같은 페이지에 다시 요청 시 모든 데이터를 다시 응답 받는 것이 아닌 **일부 캐시 된 데이터를 사용**하여 더 빠르게 웹 페이지를 렌더링

Name	Status	Type	Initiator	Size	Time	Waterfall
data:image/png;base...	200	png	main.da85589c.js?o...	(memory cache)	0 ms	
nsd94830276.png	200	png	main.da85589c.js?o...	(memory cache)	0 ms	
005.png	200	png	main.da85589c.js?o...	(memory cache)	0 ms	
029.png	200	png	main.da85589c.js?o...	(memory cache)	0 ms	
016.png	200	png	main.da85589c.js?o...	(memory cache)	0 ms	
056.png	200	png	main.da85589c.js?o...	(memory cache)	1 ms	
970.png	200	png	main.da85589c.js?o...	(memory cache)	0 ms	
021.png	200	png	main.da85589c.js?o...	(memory cache)	0 ms	
data:image/png;base...	200	png	main.da85589c.js?o...	(memory cache)	0 ms	
308.png	200	png	main.da85589c.js?o...	(memory cache)	0 ms	
data:image/png;base...	200	png	chrome-error://chro...	(memory cache)	0 ms	
804.png	200	png	main.da85589c.js?o...	(memory cache)	0 ms	

<그림1_Vue_Basic Syntax 02_Cache 예시>

computed와 method의 적절한 사용처

- computed

- 의존하는 데이터에 따라 결과가 바뀌는 계산된 속성을 만들 때 유용
- 동일한 의존성을 가진 여러 곳에서 사용할 때 계산 결과를 캐싱하여 중복 계산 방지

- method

- 단순히 특정 동작을 수행하는 함수를 정의할 때 사용
- 데이터에 의존하는지 여부와 관계없이 항상 동일한 결과를 반환하는 함수

method와 computed 정리

- computed
 - 의존된 데이터가 변경되면 자동으로 업데이트

- method
 - 호출해야만 실행됨

➤ 무조건 computed만 사용하는 것이 아니라 **사용 목적과 상황에 맞게** computed와 method를 적절히 조합하여 사용

Conditional Rendering

v-if

v-if

표현식 값의 true/false를 기반으로
요소를 조건부로 렌더링

v-if는 특정 조건이 참(true)일 때만 HTML 요소를 화면에 보여주도록 하는 Directive 입니다.
조건이 거짓(false)이면, 해당 요소는 DOM(문서 구조)에서 완전히 제거되어 보이지 않게 됩니다.
주로 사용자의 로그인 상태에 따라 다른 메뉴를 보여주거나, 특정 상황에만 경고 메시지를 표시하는 등
조건부 렌더링에 사용됩니다.

v-if

- 'v-if' Directive를 사용하여 조건부로 블록을 렌더링

```
<!-- conditional-rendering.html -->
```

```
const isSeen = ref(true)
```

```
<p v-if="isSeen">true일때 보여요</p>
```

true일때 보여요

토글

<그림2_Vue_Basic Syntax 02_v-if 예시 결과>

※ 토글을 눌러서 isSeen이 True가 된 경우



Directive

'v-' 접두사가 있는 특수 속성

<개념4_Vue_Basic Syntax 02_Directive>

v-else

- 'v-if' Directive를 사용하여 조건부로 렌더링

```
<!-- conditional-rendering.html -->
```

```
const isSeen = ref(true)
```

```
<p v-if="isSeen">true일때 보여요</p>
```

```
<p v-else>false일때 보여요</p>
```

```
<button @click="isSeen = !isSeen">토글</button>
```

true일때 보여요

토글

false일때 보여요

토글

<그림3_Vue_Basic Syntax 02_v-else 예시 결과>

v-else-if

- 'v-else-if' directive를 사용하여 v-if에 대한 else if 블록을 나타낼 수 있음
- name에 할당된 값을 바꾸면 조건에 맞는 태그가 보임

```
const name = ref('Cathy')
```

```
<div v-if="name === 'Alice'">Alice입니다</div>  
<div v-else-if="name === 'Bella'">Bella입니다</div>  
<div v-else-if="name === 'Cathy'">Cathy입니다</div>  
<div v-else>아무도 아닙니다.</div>
```


여러 요소에 대한 v-if 적용

- <template> 요소에 v-if를 사용하면, 여러 요소를 하나의 조건부 블록으로 묶을 수 있음
(v-else, v-else-if 모두 적용 가능)

```
const name = ref('Cathy')
```

```
<template v-if="name === 'Cathy'">  
  <div>Cathy입니다</div>  
  <div>나이는 30살입니다</div>  
</template>
```

```
<div>Cathy입니다</div>  
<div>나이는 30살입니다</div>
```

<그림4_Vue_Basic Syntax 02_여러 요소에 대한 v-if 적용 결과>

TIP

HTML template element

- 페이지가 로드 될 때 렌더링 되지 않지만 JavaScript를 사용하여 나중에 문서에서 사용할 수 있도록 하는 HTML을 보유하기 위한 메커니즘입니다.
- 보이지 않는 wrapper 역할을 합니다.

v-if vs v-show

v-show

v-show

표현식 값의 true/false를 기반으로
요소의 가시성을 전환

v-show는 v-if와 비슷하게 특정 조건에 따라 HTML 요소를 보여주거나 숨기는 Directive 입니다.
하지만 DOM에서 요소를 완전히 제거하는 v-if와 달리, v-show는 CSS의 display 속성을 none으로 바꿔
화면에서만 보이지 않게 숨깁니다.
요소를 자주 보여주고 숨겨야 할 경우, 렌더링 비용이 높은 v-if보다 성능적으로 유리합니다.

v-show 예시

- v-show를 사용한 요소는 조건과 관계없이 항상 DOM에 렌더링 됨
- CSS display 속성만 전환하기 때문 (none 속성으로 변환)

```
const isShow = ref(false)
```

```
<div v-show="isShow">v-show</div>
```

```
<div style="display: none;">v-show</div>
```

<그림5_Vue_Basic Syntax 02_v-show 예시 결과>



CSS display

요소를 블록, 인라인, 플렉스 등
어떤 방식으로 배치할지 정의

<개념5_Vue_Basic Syntax 02_CSS display>



none

요소를 화면에서 아예 없애,
공간조차 차지하지 않게 만들

<개념6_Vue_Basic Syntax 02_none>

v-if와 v-show의 적절한 사용처

- v-if (Cheap initial load, expensive toggle)
 - 초기 조건이 false인 경우 아무 작업도 수행하지 않음
 - 토글 비용이 높음
- v-show (Expensive initial load, cheap toggle)
 - 초기 조건에 관계 없이 항상 렌더링
 - 초기 렌더링 비용이 더 높음



렌더링
코드를 보고 실제 화면을 만드는
과정

<개념7_Vue_Basic Syntax 02_렌더링>

※ 콘텐츠를 매우 자주 전환해야 하는 경우에는 v-show를, 실행 중에 조건이 변경되지 않는 경우에는 v-if를 권장

List Rendering

v-for

v-for

소스 데이터를 기반으로 요소 또는 템플릿 블록을 반복 렌더링

※ (Array, Object, Number, String, Iterable)

v-for는 배열(Array)이나 객체(Object)의 데이터를 렌더링하는 반복문 Directive 입니다.
게시글 할 일, 상품 목록 등 동일한 구조의 요소를 여러 번 반복해서 화면에 표시할 때 사용됩니다.

v-for 구조

- v-for는 `alias in expression` 형식의 특별한 구문을 사용

```
<div v-for="item in items">
  {{ item.text }}
</div>
```

- 객체는 key-value 쌍으로 이루어져 있어, 값(value), 키(key), 인덱스(index)를 조합하여 순회할 수 있음

```
<!-- 가장 기본적인 구조-->
<div v-for="(item, index) in arr"></div>

<!-- 값만 순회 -->
<div v-for="value in object "></div>

<!-- 값과 키를 순회-->
<div v-for="(value, key) in object"></div>

<!-- 값과 키, 인덱스를 순회-->
<div v-for="(value, key, index) in object"></div>
```

v-for 예시 (1/2)

- 배열을 반복하는 예시

```
<!-- list-rendering.html -->
```

```
const myArr = ref([  
  { name: 'Alice', age: 20 },  
  { name: 'Bella', age: 21 }  
])
```

```
<div v-for="(item, index) in myArr">  
  {{ index }} / {{ item.name }}  
</div>
```

0 / Alice
1 / Bella

<그림6_Vue_Basic Syntax 02_v-for 예시 결과>

v-for 예시 (2/2)

- 객체를 반복하는 예시

```
<!-- list-rendering.html -->
```

```
const myObj = ref({  
  name: 'Cathy',  
  age: 30  
})
```

```
<div v-for="(value, key, index) in myObj">  
  {{ index }} / {{ key }} / {{ value }}  
</div>
```

```
0 / name / Cathy  
1 / age / 30
```

<그림7_Vue_Basic Syntax 02_v-for 객체 예시 결과>

여러 요소에 대한 v-for 적용

- HTML template 요소에 v-for를 사용하여 하나 이상의 요소에 대해 반복 렌더링 할 수 있음

```
<!-- list-rendering.html -->

<ul>
  <template v-for="item in myArr">
    <li>{{ item.name }}</li>
    <li>{{ item.age }}</li>
    <hr>
  </template>
</ul>
```

- Alice
- 20
- Bella
- 21

<그림8_Vue_Basic Syntax 02_여러 요소에 대한 v-for 예시 결과>

중첩된 v-for

- 각 v-for 의 하위 영역(scope)은 상위 영역에 접근 할 수 있음

```
<!-- list-rendering.html -->
```

```
const myInfo = ref([  
  { name: 'Alice', age: 20, friends: ['Bella', 'Cathy', 'Dan'] },  
  { name: 'Bella', age: 21, friends: ['Alice', 'Cathy'] }  
])
```

```
<ul v-for="item in myInfo">  
  <li v-for="friend in item.friends">  
    {{ item.name }} - {{ friend }}  
  </li>  
</ul>
```

- Alice - Bella
- Alice - Cathy
- Alice - Dan

- Bella - Alice
- Bella - Cathy

<그림9_Vue_Basic Syntax 02_중첩된 v-for 예시 결과>

v-for with key

v-for와 key

- v-for 구문은 각 요소를 Key를 활용하여 고유한 값으로 식별할 수 있음
- key는 각 항목을 고유하게 식별할 수 있는 문자열이나 숫자여야 함

```
<!-- v-for-with-key.html -->
```

```
let id = 0  
const items = ref([  
  { id: id++, name: 'Alice' },  
  { id: id++, name: 'Bella' }  
])
```

```
<div v-for="item in items" :key="item.id">  
  {{ item.name }}  
</div>
```

Alice
Bella

<그림10_Vue_Basic Syntax 02_v-for와 key 예시 결과>

내장 특수 속성 'key'의 특징

- 각 항목이 서로 구분되는 고유 식별자 역할
 - number 혹은 string으로만 사용해야 함
 - Vue의 내부 가상 DOM 알고리즘이 이전 목록과 새 노드 목록을 비교할 때 각 node를 식별하는 용도로 사용
 - key를 통해 "이 항목은 이 데이터에 해당한다"는 힌트를 줌으로써 변경 시에도 올바른 항목만 효율적으로 업데이트할 수 있음
- 반드시 v-for와 key를 함께 사용한다

TIP

왜 v-for를 사용할 때는 key를 함께 사용해야 하나요?

- 내부 컴포넌트의 상태를 일관 되게 하여 데이터의 예측 가능한 행동을 유지하기 위함입니다.
- key라는 이름표가 있어야 다른 요소와 헷갈리지 않고 정확하고 효율적으로 업데이트할 수 있습니다.

올바른 key 선택 기준

- 권장되는 key 값
 - 데이터베이스의 고유 ID
 - 항목 고유 식별자 (예: UUID)
- 피해야 할 key 값
 - 배열 인덱스(index)
 - 객체 자체



UUID

중복되지 않는 아주 긴 고유
식별 번호

<개념8_Vue_Basic Syntax 02_UUID>

v-for with v-if

v-for와 v-if 문제 상황 (1/2)

- todo 데이터 중 이미 완료된(isComplete === true) 항목만 출력하기

```
<!-- v-for-with-v-if.html -->
let id = 0
const todos = ref([
  { id: id++, name: '복습', isComplete: true },
  { id: id++, name: '예습', isComplete: false },
  { id: id++, name: '저녁식사', isComplete: true },
  { id: id++, name: '노래방', isComplete: false }
])
```

```
<ul>
  <li v-for="todo in todos" v-if="!todo.isComplete" :key="todo.id">
    {{ todo.name }}
  </li>
</ul>
```

v-for와 v-if 문제 상황 (2/2)

- todo 데이터 중 이미 처리한(isComplete === true) todo 만 출력하기

```
<ul>  
  <li v-for="todo in todos" v-if="!todo.isComplete" :key="todo.id">  
    {{ todo.name }}  
  </li>  
</ul>
```

- v-if가 더 높은 우선순위를 가지므로 v-for 범위의 todo 데이터를 v-if에서 사용할 수 없음

✖ Uncaught TypeError: Cannot read properties of undefined (reading 'isComplete')

<그림11_Vue_Basic Syntax 02_v-for와 v-if 예러 문
구>

- 동일 요소에 v-for와 v-if를 함께 사용하면 안됨
 - ※ 동일한 요소에서 v-if가 v-for보다 우선순위가 더 높기 때문
 - ※ v-if 에서의 조건은 v-for 범위의 변수에 접근할 수 없음

v-for와 v-if 해결법 2가지

1. computed 활용
2. v-for와 <template> 요소 활용



v-for와 v-if 해결법1: computed 활용

- **computed**를 활용해 이미 필터링 된 목록을 반환하여 반복하도록 설정

```
const completeTodos = computed(() => {  
  return todos.value.filter((todo) => !todo.isComplete)  
})
```

```
<ul>  
  <li v-for="todo in completeTodos" :key="todo.id">  
    {{ todo.name }}  
  </li>  
</ul>
```



computed

원본 데이터가 바뀔 때만, 값을
알아서 다시 계산하는 ref

<개념9_Vue_Basic Syntax 02_computed>

v-for와 v-if 해결법2: v-for와 <template> 요소 활용

- v-for와 template 요소를 사용하여 v-if 위치를 이동

```
<ul>
  <template v-for="todo in todos" :key="todo.id">
    <li v-if="!todo.isComplete">
      {{ todo.name }}
    </li>
  </template>
</ul>
```

Watchers

watch

watch

watch()

하나 이상의 반응형 데이터를 감시하고,
감시하는 데이터가 변경되면 콜백 함수를 호출

watch는 데이터(ref)의 변화를 지켜보다가, 값이 바뀔 때마다 지정된 콜백 함수를 실행하는 기능입니다. 새로운 값을 계산하는 computed와 달리, watch는 데이터가 바뀔 때 특정 행동(Side Effect)을 수행하기 위해 사용됩니다.

watch 구조

1. 첫번째 인자 (source)

- watch가 감시하는 대상
(반응형 변수, 값을 반환하는 함수 등)

2. 두번째 인자 (callback function)

- source가 변경될 때 호출되는 콜백 함수
 1. newValue
 - 감시하는 대상이 변화된 값
 2. oldValue (optional)
 - 감시 하는 대상의 기존 값

```
watch(source, (newValue, oldValue) => {  
    // do something  
})
```

watch 기본 동작

- count 반응형 데이터가 변경될 때마다, 그 변화를 감지하여 특정 작업을 수행
- 버튼을 누를 때마다 count 값이 바뀌고, watch는 그 변화를 “감시”하고 있다가 즉시 콜백함수를 실행

```
<!-- watcher.html -->
```

```
<button @click="count++">Add 1</button>  
<p>Count: {{ count }}</p>
```

```
const count = ref(0)  
watch(count, (newValue, oldValue) => {  
  console.log(newValue: ${newValue}, oldValue: ${oldValue})  
})
```

```
newValue: 1, oldValue: 0
```

```
newValue: 2, oldValue: 1
```

```
newValue: 3, oldValue: 2
```

<그림12_Vue_Basic Syntax 02_watch 예시 결과>

watch 예시

- 감시하는 변수에 변화가 생겼을 때 연관 데이터 업데이트하기

```
<input v-model="message">
<p>Message length: {{ messageLength }}</p>
```

```
const message = ref('')
const messageLength = ref(0)

watch(message, (newValue) => {
  messageLength.value = newValue.length
})
```

hello!

Message length: 6

<그림13_Vue_Basic Syntax 02_watch 예시 결과2>

여러 source를 감시하는 watch

- 배열을 활용하여 여러 대상을 감시할 수 있음

```
watch([foo, bar], ([newFoo, newBar], [prevFoo, prevBar]) => {  
  /* ... */  
})
```

TIP

- 여러 소스를 감시할 때, 콜백의 인자(새 값, 이전 값)도 같은 순서의 '배열'로 전달됩니다.
- 배열 속 ref(객체)의 내부까지 감시하려면, { deep: true } 옵션을 추가로 설정해야 합니다.

computed vs watch

computed와 watch

	Computed	Watchers
공통점	데이터의 변화를 감지하고 처리	
동작	의존하는 데이터 속성의 계산된 값을 반환	특정 데이터 속성의 변화를 감시하고 작업을 수행 (side-effects)
사용 목적	계산한 값을 캐싱하여 재사용 중복 계산 방지	데이터 변화에 따른 특정 작업을 수행
사용 예시	연산 된 길이, 필터링 된 목록 계산 등	DOM 변경, 다른 비동기 작업 수행, 외부 API와 연동 등

<표1_Vue_Basic Syntax 02_computed와 watch>

※ computed와 watch 모두 의존(감시)하는 원본 데이터를 직접 변경하지 않음

Lifecycle Hooks

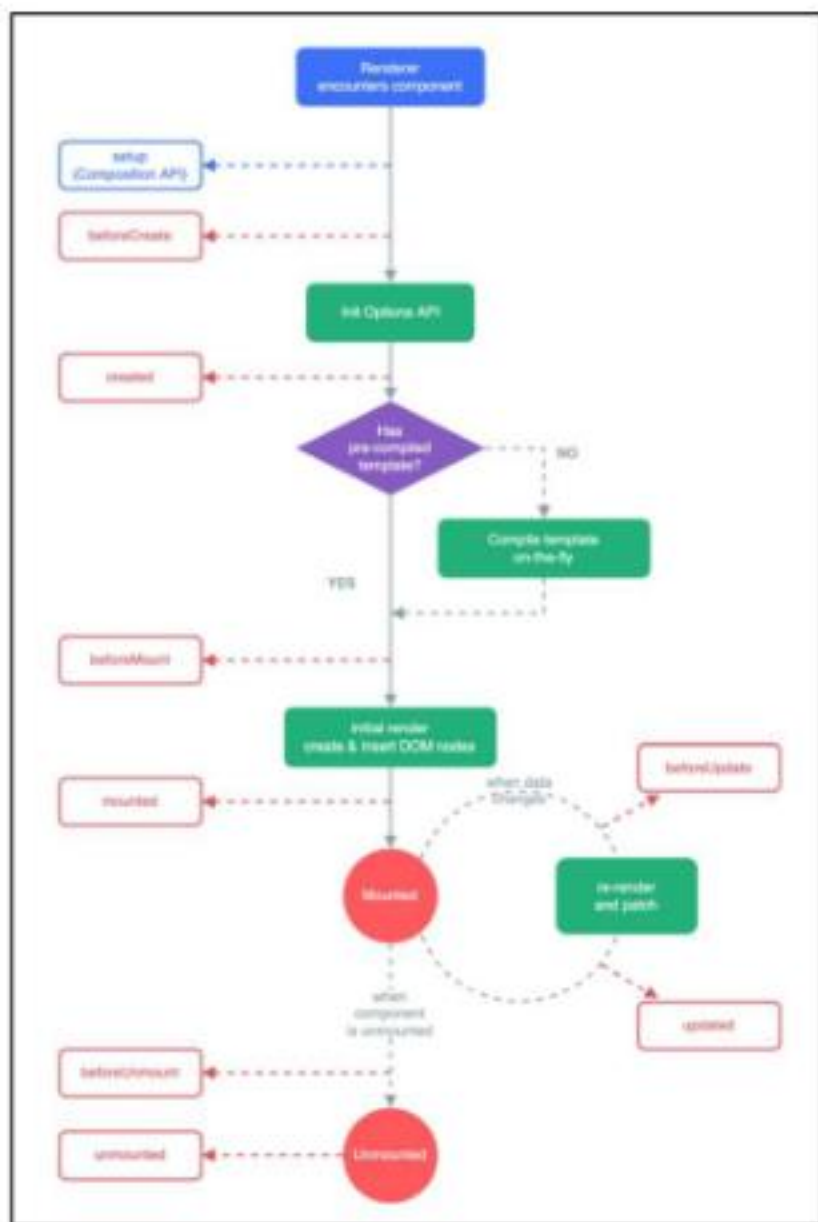
Lifecycle Hooks

- Vue 컴포넌트가 생성되고, DOM에 마운트되고, 업데이트되고, 소멸되는 각 생애 주기 단계에서 실행되도록 제공되는 함수

Lifecycle Hooks Diagram

- 컴포넌트의 생애 주기 중간 중간에 함수를 제공

※ 개발자는 컴포넌트의 특정 시점에 원하는 로직을 실행할 수 있음



<그림14_Vue_Basic Syntax 02_Lifecycle Hooks>

주요 Lifecycle Hooks

- 생성 단계 / 마운트 단계 / 업데이트 단계 / 소멸 단계 등 다양한 단계 존재
- 가장 일반적으로 사용되는 것은 `onMounted`, `onUpdated`, `onUnmounted`
- <https://vuejs.org/api/composition-api-lifecycle.html>

Composition API: Lifecycle Hooks

① Usage Note

All APIs listed on this page must be called synchronously during the `setup()` phase of a component. See [Guide - Lifecycle Hooks](#) for more details.

주요 Lifecycle Hooks: Mounting

- Vue 컴포넌트 인스턴스가 초기 렌더링 및 DOM 요소 생성이 완료된 후 특정 로직을 수행하기

```
<!-- lifecycle-hooks.html -->
```

```
const { createApp, ref, onMounted } = Vue
```

```
setup() {  
  onMounted(() => {  
    console.log('mounted')  
  })  
}
```

mounted

>

<그림16_Vue_Basic Syntax 02_Lifecycle Hooks Mounting 예시 결과>

주요 Lifecycle Hooks: Updated

- 반응형 데이터의 변경으로 인해 컴포넌트의 **DOM이 업데이트된 후** 특정 로직을 수행하기

```
<button @click="count++">Add 1</button>
<p>Count: {{ count }}</p>
<p>{{ message }}</p>
```

```
const { createApp, ref, onMounted, onUpdated } = Vue
```

```
const count = ref(0)
const message = ref(null)

onUpdated(() => {
  message.value = 'updated!'
})
```

Add 1

Count: 1

updated!

<그림17_Vue_Basic Syntax 02_Lifecycle Hooks Updated 예시 결과>

Lifecycle Hooks 활용

Lifecycle hooks with Cat API

- Mounting 시점에 Cat api에 요청을 보내고 애플리케이션 시작하기

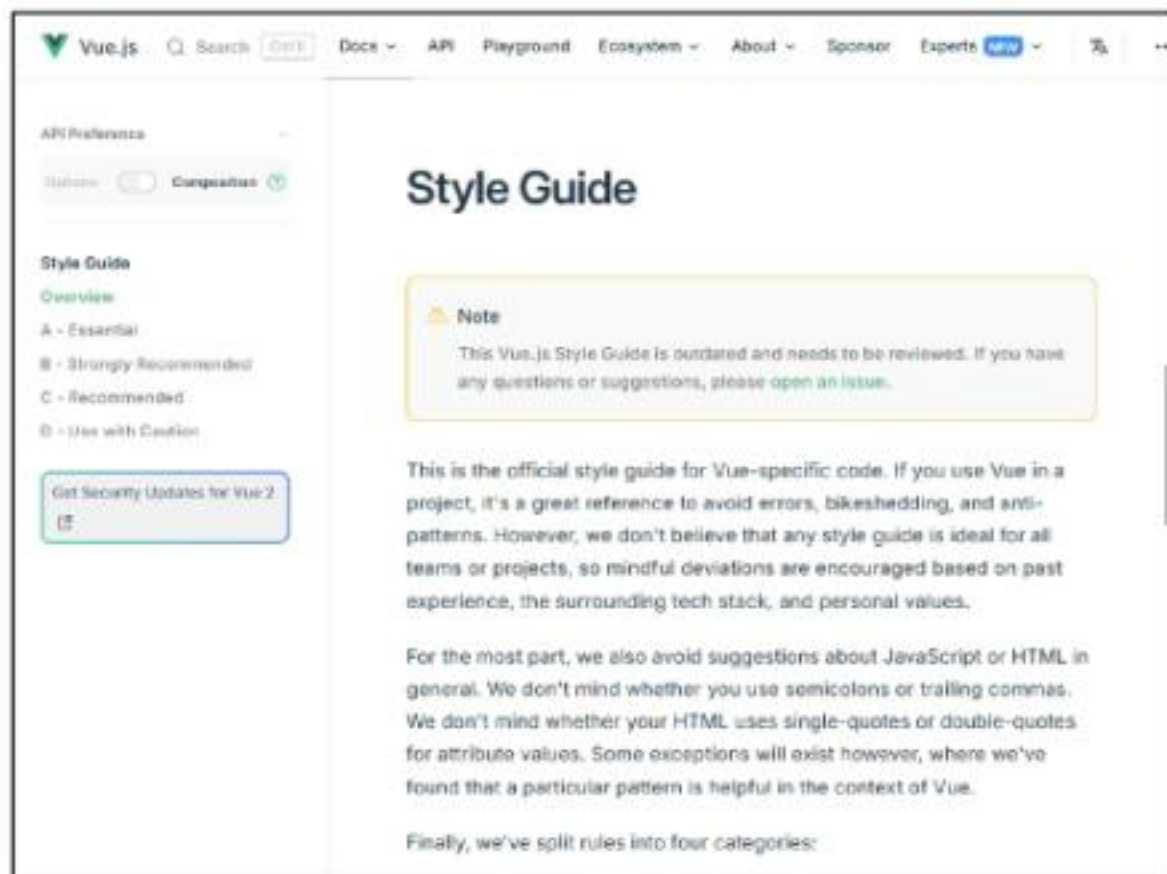
```
const getCatImage = function () {  
  axios({  
    method: 'get',  
    url: URL,  
  })  
  .then((response) => {  
    imgUrl = response.data[0].url  
    return imgUrl  
  })  
  .then((imgData) => {  
    imgElem = document.createElement('img')  
    imgElem.setAttribute('src', imgData)  
    document.body.appendChild(imgElem)  
  })  
  .catch((error) => {  
    console.log('실패했다옹')  
  })  
}
```

```
onMounted(() => {  
  getCatImage()  
})
```

Vue Style Guide

Vue Style Guide

- Vue의 스타일 가이드 규칙은 우선순위에 따라 4가지 범주로 나뉨
- 규칙 범주
 - 우선순위 A: 필수 (Essential)
 - 우선순위 B: 적극 권장 (Strongly Recommended)
 - 우선순위 C: 권장 (Recommended)
 - 우선순위 D: 주의 필요 (Use with Caution)
- <https://vuejs.org/style-guide/>



〈그림18_Vue_Basic Syntax 02_Vue Style Guide 링크 페이지〉

우선순위 별 특징

- A: 필수 (Essential)
 - 오류를 방지하는 데 도움이 되므로 어떤 경우에도 규칙을 학습하고 준수
- B: 적극 권장 (Strongly Recommended)
 - 가독성 및 개발자 경험을 향상시킴
 - 규칙을 어겨도 코드는 여전히 실행되겠지만, 정당한 사유가 있어야 규칙을 위반할 수 있음
- C: 권장 (Recommended)
 - 일관성을 보장하도록 임의의 선택을 할 수 있음
- D: 주의 필요 (Use with Caution)
 - 잠재적 위험 특성을 고려함

우선순위 A였던 금일 학습 내용

1. v-for에 key 작성하기
2. 동일 요소에 v-if와 v-for 함께 사용하지 않기

Use keyed v-for

`key` with `v-for` is *always* required on components, in order to maintain internal component state down the subtree. Even for elements though, it's a good practice to maintain predictable behavior, such as **object constancy** in animations.

<그림19_Vue_Basic Syntax 02_v-for 주의사항1>

Avoid v-if with v-for

Never use `v-if` on the same element as `v-for`.

There are two common cases where this can be tempting:

- To filter items in a list (e.g. `v-for="user in users" v-if="user.isActive"`). In these cases, replace `users` with a new computed property that returns your filtered list (e.g. `activeUsers`).
- To avoid rendering a list if it should be hidden (e.g. `v-for="user in users" v-if="shouldShowUsers"`). In these cases, move the `v-if` to a container element (e.g. `ul`, `ol`).

<그림20_Vue_Basic Syntax 02_v-for 주의사항2>

참고

computed 주의사항

1. computed의 반환 값은 변경하지 말 것

- computed의 반환 값은 의존하는 데이터의 파생된 값
 - 이미 의존하는 데이터에 의해 계산이 완료된 값
- 일종의 snapshot이며 의존하는 데이터가 변경될 때만 새 snapshot이 생성됨
- 계산된 값은 읽기 전용으로 취급되어야 하며 변경되어서는 안됨
- 대신 새 값을 얻기 위해서는 의존하는 데이터를 업데이트 해야 함

TIP

- computed 값에 직접 값을 할당하면, 기본적으로 경고가 발생하며 값이 변경되지 않습니다.

2. computed 사용 시 원본 배열 변경하지 말 것

- computed에서 reverse()나 sort()처럼 원본 배열을 변경하는 메서드를 사용할 때는, 반드시 원본 배열의 복사본을 만들어 처리해야 함

- 옳지 않은 예

```
return numbers.reverse()
```

- 옳은 예

```
return [...numbers].reverse()
```

TIP

- computed 값에 직접 값을 할당하면, 기본적으로 경고가 발생하며 값이 변경되지 않습니다.

Lifecycle Hooks 주의사항

Lifecycle Hooks 주의사항 (1/2)

- Lifecycle Hooks는 반드시 **동기적**으로 작성해야 함
- Vue는 컴포넌트가 초기화될 때 모든 Hooks를 한 번에 스캔하고 준비하기 때문
- 만약 비동기로(예: setTimeout) 훅을 등록하려고 하면, 이미 라이프사이클 단계가 지나간 후에 hooks를 설정하는 상황이 발생

```
// 비동기적으로 작성한 lifecycle hook 예시
setTimeout(() => {
  onMounted(() => {
    console.log('이 코드는 실행되지 않습니다!')
  })
}, 100)
```



동기

하나의 작업이 끝날 때까지, 다음 작업이 순서대로 기다리는 방식

<개념10_Vue_Basic Syntax 02_동기>



비동기

하나의 작업이 끝날 때까지 기다리지 않고, 다른 작업을 실행

<개념11_Vue_Basic Syntax 02_비동기>

Lifecycle Hooks 주의사항 (2/2)

- 비동기로 작성할 경우 Vue는 해당 훅을 인식하지 못하며, 원래 의도한 타이밍에 실행되지 않게 됨
- Lifecycle Hooks는 컴포넌트 로딩 과정에서 동기적으로 정의함으로써, Vue가 올바른 타이밍에 해당 로직을 수행할 수 있도록 보장해야 함

```
// 비동기적으로 작성한 lifecycle hook 예시
setTimeout(() => {
  onMounted(() => {
    console.log('이 코드는 실행되지 않습니다!')
  })
}, 100)
```

v-for와 배열을 활용한 “필터링/정렬”

v-for와 배열을 활용한 "필터링/정렬" 활용 (1/3)

- 원본 데이터를 수정하거나 교체하지 않고 필터링하거나 정렬된 새로운 데이터를 표시하는 방법
 1. computed 활용
 2. method 활용 (computed가 불가능한 중첩된 v-for에 경우 사용)

v-for와 배열을 활용한 "필터링/정렬" 활용 (2/3)

1. computed 활용

- 원본 기반으로 필터링 된 새로운 결과를 생성

```
const numbers = ref([1, 2, 3, 4, 5])

const evenNumbers = computed(() => {
  return numbers.value.filter((number) => number % 2 === 0)
})
```

```
<li v-for="number in evenNumbers">
  {{ number }}
</li>
```

- 2
- 4

<그림21_Vue_Basic Syntax 02_v-for 활용 결과1>

v-for와 배열을 활용한 "필터링/정렬" 활용 (3/3)

2. method 활용

- computed가 불가능한 중첩된 v-for에 경우 / 매개변수가 필요한 경우

```
const numberSets = ref([
  [1, 2, 3, 4, 5],
  [6, 7, 8, 9, 10]
])

const evenNumbers = function (numbers) {
  return numbers.filter((number) => number % 2 === 0)
}
```

```
<ul v-for="numbers in numberSets">
  <li v-for="num in
evenNumbers(numbers)">{{ num }}</li>
</ul>
```

- 2
- 4
- 6
- 8
- 10

<그림22_Vue_Basic Syntax 02_v-for 활용 결과2>

배열 변경 관련 메서드

배열 변경 관련 메서드

- v-for와 배열을 함께 사용 시 배열의 메서드를 주의해서 사용해야 함

1. 변화 메서드

- 호출하는 원본 배열을 변경
- `push()`, `pop()`, `shift()`, `unshift()`, `splice()`, `sort()`, `reverse()`

2. 배열 교체

- 원본 배열을 수정하지 않고 항상 새 배열을 반환
- `filter()`, `concat()`, `slice()`

Todo 애플리케이션 구현

Todo 애플리케이션 구현

- v-model, v-on, v-bind, v-for를 활용

```
<form @submit.prevent="addTodo">
  <input v-model="newTodo">
  <button>Add Todo</button>
</form>
<ul>
  <li v-for="todo in todos" :key="todo.id">
    {{ todo.text }}
    <button @click="removeTodo(todo)">X</button>
  </li>
</ul>
```

CODING

Add Todo

- Learn HTML ☐
- Learn JS ☐
- Learn Vue ☐
- SSAFY ☐

```
let id = 0
```

```
const newTodo = ref(null)
```

```
const todos = ref([
  { id: id++, text: 'Learn HTML' },
  { id: id++, text: 'Learn JS' },
  { id: id++, text: 'Learn Vue' }
])
```

```
// 새로운 Todo 항목을 추가하는 함수
```

```
const addTodo = function () {
  // todos 배열에 새로운 Todo 객체 추가
  todos.value.push({ id: id++, text: newTodo.value })
  // 입력 필드 초기화
  newTodo.value = null
}
```

```
// 선택한 Todo 항목을 삭제하는 함수
```

```
const removeTodo = function (selectedTodo) {
  // filter를 사용하여 선택한 Todo를 제외한 새로운 배열 생성
  todos.value = todos.value.filter((todo) => todo !== selectedTodo)
}
```

Computed

- 2810. 전시 정보 페이지 만들기
 - 조건부 데이터 입력

v-for

- 3098. Todo 애플리케이션 만들기
 - Todo 생성 및 삭제
- 3099. Todo 애플리케이션 만들기
 - Todo 목록 필터링

v-if

- 2811. 전시 정보 페이지 만들기
 - 스타일 부여
- 2812. 전시 정보 페이지 만들기
 - 다양한 상황의 조건

Watchers

- 2813. 전시 정보 페이지 만들기 - watch 활용하기

Q 각 문제에 올바른 답안을 생각해봅시다

1 computed 속성의 가장 큰 특징은?

- a) 항상 메서드를 호출함
- b) 데이터를 직접 수정함
- c) 의존된 데이터를 캐싱함
- d) 비동기 로직을 처리함

2 computed 대신 메서드를 사용해야 하는 경우는?

- a) 데이터 캐싱이 필요할 때
- b) 템플릿을 간결하게 할 때
- c) 계산에 인자가 필요할 때
- d) 반응형 데이터가 없을 때

3 v-if 디렉티브의 설명으로 올바른 것은?

- a) CSS display 속성을 변경한다.
- b) 조건이 거짓이면 DOM에서 제거된다.
- c) 항상 DOM에 렌더링된다.
- d) 토글 비용이 매우 저렴하다.

4 v-if와 v-show 중 자주 토글할 때 더 효율적인 것은?

- a) v-if
- b) v-show
- c) 둘 다 동일함
- d) 상황에 따라 다름

Q 각 문제에 올바른 답안을 생각해봅시다

5 v-for 사용 시 함께 작성해야 하는 속성은?

- a) id
- b) class
- c) key
- d) index

7 v-for로 렌더링할 목록을 필터링하는 가장 좋은 방법은?

- a) v-if를 함께 사용
- b) computed 속성 사용
- c) watch 함수 사용
- d) methods에 함수 작성

6 v-for와 v-if를 같은 요소에 쓰면 안 되는 이유는?

- a) v-for의 우선순위가 더 높아서
- b) v-if의 우선순위가 더 높아서
- c) 성능이 크게 저하되어서
- d) 에러가 발생하지 않아서

8 watch 함수의 주된 용도는 무엇인가요?

- a) 데이터 값을 계산하고 반환
- b) 데이터 변경 시 특정 작업 수행
- c) 템플릿의 가독성 향상
- d) DOM 요소를 직접 조작

Q 각 문제에 올바른 답안을 생각해봅시다

9 computed와 watch의 가장 큰 차이점은?

- a) 캐싱 여부
- b) 반환 값 유무
- c) 비동기 처리 가능 여부
- d) 의존 데이터 개수

10 컴포넌트가 DOM에 마운트된 후 호출되는 훅은?

- a) onCreated
- b) onUpdated
- c) onMounted
- d) onBeforeMount

11 라이프사이클 훅은 어떻게 작성해야 하나요?

- a) 비동기적으로 작성
- b) 동기적으로 작성
- c) setTimeout 안에 작성
- d) watch 안에 작성

12 computed에서 원본 배열을 정렬할 때 주의할 점은?

- a) 원본 배열을 직접 수정해야 함
- b) 복사본을 만들어 수정해야 함
- c) watch를 대신 사용해야 함
- d) 정렬은 불가능함

확인 문제



정답 및 해설

1. c) 의존된 데이터를 캐싱함 2. c) 계산에 인자가 필요할 때 3. b) 조건이 거짓이면 DOM에서 제거된다. 4. b) v-show
5. c) key 6. b) v-if의 우선순위가 더 높아서 7. b) computed 속성 사용 8. b) 데이터 변경 시 특정 작업 수행

1. 의존하는 데이터가 바뀌기 전까지 계산된 결과를 저장(캐싱)하여 성능을 향상시킵니다.
2. computed는 인자를 받을 수 없으므로, 계산에 외부 값이 필요하다면 메서드를 사용해야 합니다.
3. v-if는 조건이 거짓일 경우, 해당 요소를 DOM 트리에서 완전히 제거하여 렌더링하지 않습니다.
4. v-show는 CSS display 속성만 바꾸므로, 자주 변경될 때 렌더링 비용이 더 저렴합니다.

5. key는 각 노드를 고유하게 식별하여 효율적으로 업데이트하기 위해 반드시 필요합니다.
6. v-if가 먼저 평가되므로, v-for의 변수에 접근할 수 없어 에러가 발생할 수 있습니다.
7. computed를 사용하여 미리 필터링된 배열을 만들면 템플릿이 간결하고 효율적입니다.
8. 데이터 변경에 대한 반응으로 비동기 요청이나 다른 부수 효과(side effect)를 수행할 때 사용됩니다.

확인 문제



정답 및 해설

9. b) 반환 값 유무 10. c) onMounted 11. b) 동기적으로 작성 12. b) 복사본을 만들어 수정해야 함

- 9. computed는 계산된 값을 반환하지만, watch는 특정 작업을 수행하고 값을 반환하지 않습니다.
- 10. onMounted는 컴포넌트의 초기 렌더링 및 DOM 노드 생성이 완료된 후에 실행됩니다.
- 11. 라이프사이클 혹은 정해진 순서대로 동기적으로 실행되는 함수입니다.
- 12. sort()나 reverse()는 원본을 변경하므로, 복사본을 만들어 사용해야 부수 효과를 방지합니다.

활동 정리

핵심 키워드

개념	설명	예시
computed 속성	의존 데이터를 기반으로 캐싱되는 값	<code>const c = computed(() => {})</code>
v-if 디렉티브	조건에 따라 요소를 렌더링/제거	<code><p v-if="isSeen">Hi</p></code>
v-show 디렉티브	조건에 따라 CSS display 전환	<code><p v-show="isShow">Hi</p></code>
v-for 디렉티브	데이터를 기반으로 목록을 렌더링	<code><li v-for="i in items"></code>
key 속성	v-for 각 노드를 식별하는 고유 값	<code><li :key="item.id"></code>
watch 함수	데이터 변경을 감지하고 콜백 실행	<code>watch(count, (newV) => {})</code>
라이프사이클 훅	컴포넌트 생애 주기 특정 시점 함수	<code>onMounted(() => {})</code>

<표2_Vue_Basic Syntax 02_핵심키워드>

활동 정리

오늘 공부한 내용

요약 및 정리

- computed()
 - 반응형 데이터를 기반으로 하는 계산된 속성을 만드는 함수
 - 템플릿 안의 표현식이 너무 복잡해지는 것을 방지하고, 재사용성을 높임
 - 가장 큰 특징은 캐싱(caching)
 - 의존하는 반응형 데이터가 변경될 때만 값을 다시 계산하고, 그렇지 않으면 이전에 계산된 값을 즉시 반환하여 성능을 향상
 - 메서드(Methods)는 호출될 때마다 항상 함수를 실행하지만, computed는 의존 데이터가 바뀔 때만 실행된다는 점에서 차이

활동 정리

오늘 공부한 내용 요약 및 정리

- v-if

- 조건이 true일 때만 블록을 렌더링하고, 조건이 false이면 해당 요소는 DOM에서 완전히 제거
- v-else, v-else-if와 함께 사용 가능

- v-show

- 조건에 따라 요소의 가시성을 전환하지만, 항상 DOM에 렌더링된 상태를 유지
- 단순히 CSS display 속성을 none으로 변경하여 숨김

- v-if vs v-show

- 자주 토글해야 하는 요소에는 v-show가 더 효율적
- 런타임 중에 조건이 거의 바뀌지 않는다면 v-if가 더 적합함

활동 정리

오늘 공부한 내용

요약 및 정리

- v-for
 - 배열이나 객체를 기반으로 목록을 반복해서 렌더링하는 디렉티브
 - key 속성
 - v-for를 사용할 때는 각 항목을 고유하게 식별할 수 있는 key를 반드시 함께 제공해야 함
 - Vue는 이 key를 사용해 변경된 항목을 효율적으로 추적하고 업데이트
- v-for와 v-if
 - v-if의 우선순위가 더 높기 때문에 두 디렉티브를 같은 요소에 함께 사용하면 안 됨
 - 해결책
 - computed 속성으로 미리 필터링된 배열을 생성
 - <template> 태그를 사용해 v-if의 위치를 조정

활동 정리

오늘 공부한 내용

요약 및 정리

- watch
 - 특정 반응형 데이터를 감시하고, 데이터가 변경될 때마다 지정된 콜백 함수를 실행하는 기능
 - computed가 데이터 변경에 따라 새로운 값을 계산하는 데 사용되지만
 - watch는 데이터 변경에 대한 반응으로 비동기 요청이나 다른 특정 작업(side effect)을 수행할 때 주로 사용

활동 정리

오늘 공부한 내용

요약 및 정리

- Lifecycle Hooks

- Vue 컴포넌트가 생성되고, DOM에 추가되고, 업데이트되고, 제거되는 각 생애 주기 단계에서 특정 로직을 실행할 수 있도록 제공되는 함수들
- onMounted
 - 컴포넌트가 DOM에 마운트된 직후에 호출되는 훅으로, 보통 서버에서 초기 데이터를 가져오는 등의 작업에 사용
- onUpdated
 - 반응형 데이터의 변경으로 인해 컴포넌트의 DOM이 업데이트된 후 특정 로직을 수행

활동 정리

Vue의 다양한 기능들은 어떤 역할을 할까요?

- 아래 퀴즈를 풀어봅시다.

1. 조건에 따라 요소를 DOM에서 완전히 제거 → **v-if**
2. 조건에 따라 요소를 보이거나 숨기기만 함 → **v-show**
3. 배열 데이터를 기반으로 목록을 반복하여 렌더링 → **v-for**
4. 데이터가 바뀔 때마다 새로운 값을 계산하고 결과를 기억(캐싱) → **computed**
5. 데이터가 바뀔 때마다 특정 행동(Side Effect)을 수행 → **watch**

감사합니다

